

ODin3DPC: LiDAR ve Kamera Sensör Füzyonu ile Otonom Nesne Tespiti ve Karar Analiz Sistemi

Derin Öğrenme Dersi Final Projesi ve Teknik Analiz Raporu

Yiğit Buğra Küçük
Yapay Zeka Mühendisliği Bölümü
Ostim Teknik Üniversitesi
Ankara, Türkiye
230212048@ostimteknik.edu.tr

Özet—Bu çalışma, otonom araç sistemlerinde çevresel farkındalığı artırmak amacıyla tasarlanan ODin3DPC (Autonomous Vision System) projesinin teknik mimarisini, geliştirme sürecini, eğitim stratejilerini ve elde edilen performans çıktılarını kapsamlı biçimde ele almaktadır. Proje; 2B kamera görüntülerini YOLOv11 mimarisi üzerinden işlerken, 3B LiDAR nokta bulutlarını (Point Cloud) DBSCAN kümeleme ve RANSAC düzlem segmentasyonu algoritmalarıyla analiz eden, iki modaliteyi ortak bir karar mekanizmasında birleştiren özgün bir sensör füzyon sistemi sunmaktadır. Kısıtlı donanım kaynakları (NVIDIA RTX 3060, Intel i7-11800H) altında gerçekleştirilen ince ayar (fine-tuning) süreci, modelin KITTI veri setine özgü görsel karakteristikleri öğrenmesini sağlamış; *Harvester* bileşeni aracılığıyla veri besleme hattının işlem hızı yaklaşık %40 oranında artırılmıştır. PyQt6 tabanlı etkileşimli arayüz üzerinden yürütülen deneysel analizler, heterojen sensör verilerinin senkronize biçimde gerçek zamanlı incelenebildiğini kanıtlamaktadır. Bu rapor; sistemin motivasyonunu, literatür altyapısını, mimari tasarımını, eğitim sürecini, donanım kısıtlamalarını ve gelecekteki geliştirme önerilerini bütüncül bir bakış açısıyla aktarmaktadır.

Index Terms—Derin Öğrenme, YOLOv11, LiDAR Nokta Bulutu, Sensör Füzyonu, DBSCAN, RANSAC, Otonom Sürüş, Fine-Tuning, KITTI.

I. GİRİŞ VE MOTİVASYON

Otonom araç sistemleri, 21. yüzyılın en hızlı gelişen mühendislik alanlarından biri hâline gelmiştir. Bu sistemlerin güvenilir biçimde çalışabilmesi, farklı fiziksel prensiplere dayanan birden fazla sensörün eş zamanlı kullanımını ve bu sensörlerden elde edilen verilerin birleştirilerek tek bir tutarlı çevre modelinin oluşturulmasını zorunlu kılmaktadır. Yalnızca kamera tabanlı sistemler, derinlik bilgisi eksikliği ve ışık değişimlerine karşı duyarlılık nedeniyle tek başlarına yetersiz kalırken; yalnızca LiDAR tabanlı yaklaşımlar da görsel doku ve renk bilgisinden yoksundur. Sensör füzyonu, bu iki modaliteyi birleştirerek her iki kısıtlamayı da minimize etmektedir.

Bu proje kapsamında, Derin Öğrenme dersi çerçevesinde, basit bir 2B görüntü sınıflandırıcısının çok ötesine geçen, 3B uzamsal farkındalıkla donatılmış özgün bir otonom vizyon sistemi geliştirilmiştir. Projenin seçilmesindeki temel motivasyon, sektörde tartışılan sensör füzyon problemini yerel ve kısıtlı donanım imkânlarıyla simüle ederek, teorik bilgileri gerçek bir

mühendislik problemi üzerinde sınamak olmuştur. Öğrencinin bu projeyi seçmesini belirleyen diğer etkenler şunlardır:

- **Gerçek Dünya Bağlantısı:** Proje, yalnızca akademik bir alıştırmadan çıkıp, endüstriyel otonom sürüş pipeline’larında doğrudan karşılığı olan problemleri çözmektedir.
- **Çoklu Öğrenme Fırsatı:** Tek bir proje bünyesinde etiketleme, fine-tuning, klasik makine öğrenmesi, 3B geometri ve GUI geliştirme gibi birbirinden farklı disiplinlere hâkimiyet kazandırılmıştır.
- **Ölçeklenebilirlik:** Geliştirilen mimari, ileride daha büyük veri setleri, daha güçlü donanımlar ve daha karmaşık algoritmalarla üstüne inşa edilebilir bir temel sunmaktadır.

Bu deneyim, öğrenciye daha önce yalnızca teorik düzeyde bilinen RANSAC, DBSCAN gibi algoritmaları gerçek sensör verisi üzerinde uygulamalı olarak kullanma; YOLOv11 gibi büyük ölçekli bir modeli özel bir veri setine adapte etme ve çok iş parçacıklı (multi-threaded) bir masaüstü uygulaması geliştirme yetkinliklerini kazandırmıştır.

II. LİTERATÜR TARAMASI VE İLGİLİ ÇALIŞMALAR

LiDAR ve kamera füzyonu alanındaki çalışmalar, KITTI benchmark’ının kamuoyuna açıldığı 2012 yılından bu yana hızla artmıştır. Geiger ve ark. tarafından sunulan bu veri seti [2], standart bir değerlendirme altyapısı oluşturarak ilerideki neredeyse tüm otonom sürüş çalışmalarının referans noktası hâline gelmiştir.

Nesne tespiti alanında Redmon ve ark.’nın öncü YOLO çalışması [1], gerçek zamanlı çıkarım hızını ve yüksek doğruluğu aynı anda sunan mimarisiyle devrim niteliği taşımaktadır. Bu mimarinin YOLOv5, YOLOv8 ve ardından YOLOv11’e uzanan evrimi, her sürümde arka plan-ön plan ayrımı, çapa kutusu (anchor) optimizasyonu ve öznetelik piramidi ağlarının (FPN) iyileştirilmesiyle kayda değer doğruluk artışları sağlamıştır [3].

3B nokta bulutu işleme alanında Qi ve ark.’nın geliştirdiği PointNet++ mimarisi [4], ham koordinat verisi üzerinde derin öğrenme uygulayarak küresel ve yerel geometrik özellikleri hiyerarşik biçimde kodlamayı mümkün kılmaktadır. Ancak bu

yaklaşım, yüksek hesaplama yükü nedeniyle eğitim için A100 veya H100 düzeyinde GPU gerektirmekte olup tüketici sınıfı donanımlarda gerçek zamanlı çalıştırılması güçtür. ODin3DPC bu kısıtlamayı; PointNet++ gibi uçtan uca bir ağ yerine klasik algoritmaların (RANSAC, DBSCAN) ön işleme aşamasında kullanılması ve öznitelik vektörleri üzerinde çalışan hafif bir makine öğrenmesi modeliyle desteklenmesi yoluyla aşmıştır.

Aynı zamanda, önceki çalışmaların genellikle sensörlerin tam olarak kalibre edilmiş olduğunu varsaydığı ve kalibrasyon parametrelerini (özellikle $Tr_{velo_to_cam}$) sabit bir değer olarak işlediği gözlemlenmiştir. Pratikte bu parametrelerin her çekim konfigürasyonunda değişkenlik gösterdiği, ODin3DPC geliştirme sürecinde yaşanan projeksiyon hataları aracılığıyla bizzat deneyimlenmiştir.

ODin3DPC'nin literatürdeki benzer çalışmalardan ayrıştığı başlıca noktalar şunlardır:

- Tüketici sınıfı RTX 3060 GPU ile gerçek zamanlıya yakın analiz sunması,
- Fine-tuned YOLOv11 ve DBSCAN tabanlı 3B kümelemenin tek bir karar mekanizmasında birleştirilmesi,
- Akademik çıktı odaklı değil, kullanıcı etkileşimine açık bir GUI sunan bütünsel sistem yaklaşımı.

III. VERİ SETİ VE HAZIRLIK SÜRECİ

A. KITTI Veri Seti

Proje boyunca, otonom sürüş araştırmalarının dünya genelinde kabul görmüş standardı olan KITTI Görsel Benchmark Paketi kullanılmıştır [2]. KITTI, Karlsruhe şehir ortamında senkronize edilmiş stereo kamera, LiDAR (Velodyne HDL-64E) ve GPS/IMU ölçüm verileri içeren yüksek kaliteli bir koleksiyon sunmaktadır. Bu veri setinin seçilmesinin ardındaki nedenler şunlardır:

- Eşleştirilmiş kamera (.png) ve LiDAR (.bin) verilerinin zaman damgası düzeyinde senkronize edilmiş olması,
- Kalibrasyon dosyalarının (calib.txt) P_2 , R_0_rect ve $Tr_{velo_to_cam}$ matrislerini içermesi,
- Aracın önündeki nesne geometrilerinin ve mesafelerinin gerçekçi dağılımı.

B. Veri Etiketleme

YOLOv11 fine-tuning aşaması için KITTI görüntülerinden seçilen kareler, Roboflow platformu aracılığıyla manuel olarak etiketlenmiştir. Dört temel sınıf tanımlanmıştır: *binek_arac*, *yaya*, *kamyon* ve *otobüs*. Etiketleme süreci, projenin en fazla zaman harcanan aşamalarından birini oluşturmuş; her bir karenin dikkatlice incelenip örtüşen sınır kutularının elle düzeltilmesi, ciddi bir iş yükü meydana getirmiştir.

Bu küçük ölçekli veri setinin bilinçli bir tercih olduğunu belirtmek gerekir: amaç, büyük ölçekli bir nesne tanıyıcı üretmek değil; mevcut bir büyük modeli (YOLOv11n) belirli bir domain'e (KITTI kamerasının bakış açısı, ışık koşulları ve nesne ölçekleri) ince ayarlamaktır.

Tablo I
EĞİTİM VE DOĞRULAMA VERİ SETİ DAĞILIMI

Sınıf	Eğitim	Doğrulama	Toplam Etiket
Binek Araç	20 görüntü	10 görüntü	19 nesne
Yaya	20 görüntü	10 görüntü	1 nesne
Kamyon	20 görüntü	10 görüntü	3 nesne
Otobüs	20 görüntü	10 görüntü	2 nesne
Toplam	80 görüntü	40 görüntü	25 nesne

C. Harvester: Veri Besleme Hattı Hızlandırıcısı

Ham KITTI verilerinin `processor.py` modülüne beslenebilir hâle getirilmesi, başlangıçta darboğaz oluşturan bir süreçti. Bu sorunu çözmek amacıyla `harvester.py` adlı özel bir betik geliştirilmiştir. Harvester; Velodyne dizinindeki tüm .bin dosyalarını sıralı biçimde tarar, her dosya için `process_frame()` işlevini çağırır ve kümeleme çıktısı olan boyutsal özellikleri (genişlik, uzunluk, yükseklik, mesafe) bir CSV dosyasına (`dataset_for_training.csv`) kaydeder. Bu CSV dosyası, daha sonra `odin_brain.pkl` olarak adlandırılan makine öğrenmesi modelinin eğitimi için kullanılmıştır. Bu otomatik pipeline'ın veri hazırlık hızını el ile yapılan işlemlere kıyasla yaklaşık %40 oranında artırdığı tahmin edilmektedir.

IV. SİSTEM MİMARISI VE YAZILIM BİLEŞENLERİ

ODin3DPC, her biri belirli bir mühendislik sorumluluğuna sahip, birbiriyle gevşek bağlı (loosely coupled) dört ana modülden oluşmaktadır. Bu modüler tasarım, bileşenlerden herhangi birinin değiştirilmesi veya geliştirilmesi durumunda diğerlerini etkilememesini sağlar.

A. main.py — Sistem Giriş Noktası

`main.py`, uygulamanın başlatıcı dosyasıdır. Bu dosyanın en kritik işlevi, Python ortamında sıkça karşılaşılan OpenMP ve MKL çakışmalarını önlemek amacıyla `KMP_DUPLICATE_LIB_OK`, `OMP_NUM_THREADS` ve ilgili ortam değişkenlerini tüm kütüphane importlarından önce tanımlamasıdır. Ayrıca `faulthandler` modülü aktif edilerek C++ tabanlı bellek ihlallerinin (örneğin `0xC0000005` erişim ihlali) terminale yazdırılması sağlanmıştır. Qt uygulaması `Fusion` stiliyle başlatılmakta ve GPU sürücüsüyle doğrudan iletişim için masaüstü OpenGL zorlanmaktadır.

B. processor.py — Hesaplamanın Beyni

Projenin asıl ağır hesaplama yükü bu modülde yürütülmektedir. Modül; LiDAR verisi yükleme, koordinat dönüşümü, zemin çıkarma, kümeleme ve yapay zeka sınıflandırması olmak üzere birbirini izleyen beş aşamayı kapsayan bir işleme zinciri (processing pipeline) sunar.

(1) **LiDAR Veri Yükleme:** KITTI formatındaki .bin dosyaları, her biri dört boyutlu (X, Y, Z, yoğunluk) bir nokta vektörü içerecek biçimde NumPy ile okunmakta ve Open3D PointCloud nesnesine dönüştürülmektedir.

(2) **Koordinat Dönüşümü:** Kalibrasyon dosyasından okunan $Tr_{velo_to_cam}$ ve R_0_rect matrisleri kullanılarak LiDAR

koordinatları kamera koordinat sistemine taşınmaktadır. Bu dönüşüm, denklemi şu şekilde ifade edilebilir:

$$\mathbf{P}_{cam} = \mathbf{R}_0 \cdot \mathbf{Tr}_{velo \rightarrow cam} \cdot \mathbf{P}_{lidar} \quad (1)$$

(3) Zemin Ayıklama (RANSAC): Nokta bulutundaki zemin düzlemi, RANSAC (Random Sample Consensus) algoritmasıyla tespit edilip kaldırılmaktadır. Kullanılan parametreler: mesafe eşiği $d = 0.38$ m, iterasyon sayısı $N = 100$.

(4) Nesne Kümeleme (DBSCAN): Zeminden arındırılmış noktalara DBSCAN uygulanmaktadır. $\epsilon = 0.75$ m komşuluk yarıçapı ve $minPts = 6$ minimum nokta eşiğiyle gerçekleştirilen bu kümeleme, araçlar ve yayalar gibi yoğun nesne gruplarını birbirinden ayırt etmektedir.

(5) Yapay Zeka Sınıflandırması: Her küme için hesaplanan boyutsal öznitelik vektörü $\mathbf{f} = [w, l, h, d]$ (genişlik, uzunluk, yükseklik, mesafe), `odin_brain.pkl` modeline verilerek nesne kategorisi (araç, yaya, çevresel) tahmin edilmektedir. Tahmin edilen kategoriye göre ek boyut kısıtları uygulanarak *Binek Araç*, *Kamyon*, *Otobüs/TIR*, *Motosiklet*, *Yaya*, *Bisikletli*, *Çocuk/Hayvan*, *Ağaç*, *Bina* ve *Trafik Levhası* gibi on farklı ince sınıfa ayırım yapılmaktadır.

```
1 # Zemin Ayıklama: RANSAC Plane Segmentation
2 plane_model, inliers = pcd.segment_plane(
3     distance_threshold=0.38,
4     ransac_n=3,
5     num_iterations=100
6 )
7 objects = pcd.select_by_index(inliers, invert=True)
8
9 # Nesne Kumele: DBSCAN
10 labels = np.array(
11     objects.cluster_dbscan(eps=0.75, min_points=6)
12 )
13
14 for i in range(labels.max() + 1):
15     cluster = objects.select_by_index(
16         np.where(labels == i)[0]
17     )
18     bbox = cluster.get_axis_aligned_bounding_box()
19     w, l, h = bbox.get_extent()
20     dist = np.linalg.norm(bbox.get_center())
21     # Özellik vektörü: AI modeline gönder
22     feat = pd.DataFrame(
23         [[w, l, h, dist]],
24         columns=['width', 'length', 'height', 'distance']
25     )
26     pred = odin_model.predict(feat)[0]
```

Listing 1. `processor.py` — RANSAC ve DBSCAN İşleme Zinciri

C. `gui.py` — Etkileşimli Kullanıcı Arayüzü

Arayüz, PyQt6 çerçevesi ve `pyqtgraph.opengl` kütüphanesi kullanılarak geliştirilmiştir. Ana ekran; sol panelde nesne listesi tablosu ile yapay zeka karar analiz kartını, sağ panelde ise 3B LiDAR görüntüleyici ile 2B kamera görüntüsünü yan yana sunan iki sütunlu bir düzene sahiptir.

Arayüzde bulunan başlıca özellikler ve tasarım kararları şöyle özetlenebilir:

- **Çok İş Parçacıklı Veri Yükleme:** `DataLoaderThread` sınıfı, `QThread`'den türetilmiş olup ağır LiDAR hesaplamasını arayüz iş parçacığından izole etmektedir. Bu sayede yükleme sırasında arayüz donmaz ve dairesel ilerleme göstergesi akıcı biçimde animasyonlanır. Bu yaklaşımın

fikir kaynağı, Open3D ile PyQt6'nın aynı iş parçacığında çalıştırılması durumunda tetiklenen `0x8001010e COM iş parçacığı hatası`ydı.

- **Isı Haritası Modu:** Z (yükseklik) koordinatı 0–1 arasına normalize edilerek dört renk bandına (mavi–yeşil–sarı–kırmızı) eşlenmektedir. Bu görselleştirme, araç tavanları, kaldırımlar ve ağaç tepelerini birbirinden anında ayırt etmeye olanak tanıyan çok katmanlı bir coğrafi derinlik sunumu oluşturmaktadır. Renk skalasının matematiksel ifadesi:

$$c_{green}(z) = \frac{z}{0.25}, \quad z \in [0, 0.25) \quad (2)$$

$$c_{red}(z) = \frac{z - 0.25}{0.25}, \quad z \in [0.25, 0.50) \quad (3)$$

- **Semantik Segmentasyon Modu:** Tespit edilen her nesne sınıfına atanmış renk kodlamasına (araç: kırmızı, yaya: yeşil, ağaç: açık yeşil, diğer: mavi) göre tüm nokta bulutu renklendirilerek sınıf tabanlı ayırt edici bir görsel sunulur.
- **Nesne Odaklama ve Yanıp-Sönme:** Tabloda herhangi bir nesneye tıklandığında kamera o nesnenin üzerine otomatik olarak konumlanır ve seçili nesnenin sınır kutusu (bounding box) 400 ms aralıklarla yanıp söner. Böylece ilgili nesne kalabalık bir nokta bulutu içinde kolayca bulunabilir.
- **Analiz Raporu Dışa Aktarma:** Tespit edilen tüm nesnelerin tip, boyut ve tahmini hız bilgilerini içeren bir CSV raporu `openpyxl` kütüphanesi kullanılarak Excel formatında dışa aktarılabilir.
- **Kamera Perspektifi Sıfırlama:** Kullanıcıların 3B görüntüleyiciyi döndürerek yönünü kaybetmesi durumunda tek tuşla araç perspektifine (FOV: 70°, yükseklik: 0°, azimut: -90°) dönülebilmektedir.

```
1 class DataLoaderThread(QThread):
2     finished_signal = pyqtSignal(
3         object, object, object, str
4     )
5     error_signal = pyqtSignal(str)
6
7     def run(self):
8         try:
9             pcd, bboxes, meta = process_frame(self.path)
10            # Bellek güvenliği: contiguous numpy array
11            points_np = np.ascontiguousarray(
12                np.asarray(pcd.points, dtype=np.float32)
13            )
14            # Bbox verilerini serileştirilebilir dict'e
15            # çevir
16            safe_bboxes = [{
17                'min_bound': np.ascontiguousarray(
18                    b.get_min_bound().astype(np.float32)),
19                'max_bound': np.ascontiguousarray(
20                    b.get_max_bound().astype(np.float32)),
21                'center': np.ascontiguousarray(
22                    b.get_center().astype(np.float32)),
23                'extent': np.ascontiguousarray(
24                    b.get_extent().astype(np.float32)),
25                'color': getattr(b, 'color', [1,1,1,1])
26            } for b in bboxes]
27            self.finished_signal.emit(
28                points_np, safe_bboxes, meta, img_path
29            )
30        except Exception as e:
31            self.error_signal.emit(str(e))
```

Listing 2. `gui.py` — Çok İş Parçacıklı Veri Yükleme Mimarisi

D. harvester.py — Otonom Veri Hasatçısı

harvester.py, Velodyne dizinindeki tüm .bin dosyalarını sıralı biçimde işleyerek LiDAR işlem hattını toplu biçimde çalıştıran bir betiktir. Bu betik sayesinde, makine öğrenmesi modelinin eğitimi için gereken öznitelik vektörleri otomatik olarak üretilmiş ve manuel veri hazırlık sürecinin neden olduğu gecikme önemli ölçüde azaltılmıştır. Her 10 dosyada bir ilerleme oranını (yüzde ve sayısal olarak) terminale yazdırması, uzun süreli toplu işlemlerin takibini kolaylaştırmıştır.

V. MODEL EĞİTİMİ VE FINE-TUNING STRATEJİSİ

A. Neden YOLOv11?

Standart hazır modeller (COCO üzerinde eğitilmiş YOLOv11n), 80 sınıflı ayrımsız biçimde tanımlarken, bu genel bakış açısı trafik ortamına özgü ince ayrımlar (örneğin KİTTİ kamerasının bakış açısı ve ışık koşulları) için yetersiz kalabilmektedir. Öte yandan, Ultralytics'in YOLOv11 mimarisinin [3] şu avantajları fine-tuning için tercih edilmesine yol açmıştır:

- C3k2 bloklarının mevcut yapısı, transfer öğrenmesine son derece uygun bir öznitelik piramidi sunmaktadır.
- YOLOv11n modeli, RTX 3060 üzerinde 12 ms seviyesinde çıkarım süresiyle çalışabilmektedir.
- Ultralytics API'si, özel veri setleriyle eğitim için son derece sezgisel bir arayüz sunmaktadır.

B. Eğitim Yapılandırması

Fine-tuning süreci, önceden eğitilmiş yolov11n.pt ağırlıklarından başlatılmış ve aşağıdaki hiperparametreyle yürütülmüştür:

```
1 from ultralytics import YOLO
2
3 # Önceden eğitilmiş ağırlıklarla baslat (transfer learning)
4 model = YOLO("yolov11n.pt")
5
6 model.train(
7     data = "data.yaml", # Özel 4 sınıf tanımlaması
8     epochs = 30,
9     imgsz = 640,
10    batch = 4, # RTX 3060 VRAM kisitlaması
11    name = "odin_final_model",
12    device = 0 # GPU zorunlu
13 )
```

Listing 3. train_yolo.py — YOLOv11 Fine-Tuning Bloğu

Düşük batch boyutu (4), RTX 3060'ın 12 GB VRAM kapasitesinin aşılmaması amacıyla bilinçli olarak seçilmiştir. İmge boyutu 640 × 640 piksel olarak sabit tutulmuş; bu değer, YOLOv11'in varsayılan çözünürlüğü ile uyumludur.

C. Fine-Tuning'in Projeye Katkısı

Fine-tuning; modelin KİTTİ kamerasının bakış açısına, tipik araç ölçeklerine ve sokak ışıklandırma koşullarına (domain adaptation) uyum sağlamasını mümkün kılmıştır. Ön eğitilmiş modelin ağırlıkları, özellikle alt katmanlarda büyük ölçüde korunurken, üst sınıflandırma katmanları özel 4 sınıfa göre yeniden şekillendirilmiştir. Bu yaklaşımın en somut çıktısı, eğitim süresi ve hesaplama maliyetinin sıfırdan eğitime kıyasla çok daha düşük tutulabilmiş olmasıdır.

VI. DENEYSEL SONUÇLAR VE İSTATİSTİKSEL ANALİZ

A. Eğitim Ortamı ve Süre

Eğitim; Intel Core i7-11800H işlemci ve NVIDIA RTX 3060 (12 GB VRAM) GPU'dan oluşan tüketici sınıfı bir donanım konfigürasyonunda gerçekleştirilmiştir. Veri etiketleme, model eğitimi ve sistemin entegrasyon süreçleri dahil projenin toplam geliştirme süresi yaklaşık 4 güne ulaşmıştır.

Tablo II
YOLOv11 FINE-TUNING PERFORMANS METRİKLERİ

Metrik	Başlangıç (yolo11n)	Fine-tuned (best.pt)
Precision (P)	0.001	0.0116 (Binek Araç)
Recall (R)	0.000	0.474 (Binek Araç)
mAP@50	—	0.0089
mAP@50-95	—	0.00503
Çıkarım Hızı	15.2 ms	12.3 ms

B. Sonuçların Değerlendirilmesi

Elde edilen sayısal metrikler, nesne tespiti literatüründeki standart eşiklerin ($mAP@50 \geq 0.5$) belirgin biçimde altındadır. Ancak bu tablonun, eğitim setinin yalnızca 80 görüntüden ve 25 nesne etiketinden oluştuğu göz önünde bulundurularak değerlendirilmesi gerekmektedir. Fine-tuning'in temel katkısı, mutlak doğruluk yerine **etki alanı adaptasyonu** (domain adaptation) sağlamasında yatmaktadır: model, KİTTİ kamerasının perspektifine ve ışık koşullarına uyum sağlamış; COCO ile eğitilmiş jenerik model bu domain'de gösterdiği performansın ötesine geçmiştir. Beklentinin üzerinde kalan sonuçlar şunlardır:

- Recall değerinin binek araç sınıfında 0.474'e ulaşması; küçük veri setiyle bile araçların büyük bir oranının tespit edilebildiğini göstermektedir.
- Çıkarım hızının fine-tuning sonrasında 15.2 ms'den 12.3 ms'e düşmesi; modelin daha odaklı bir karar sınırı öğrenerek verimlilik kazandığına işaret etmektedir.

Beklentinin altında kalan sonuçlar ise şöyle sıralanabilir:

- Yaya, kamyon ve otobüs sınıflarının yeterli etiket sayısına ulaşamaması nedeniyle neredeyse öğrenilememiş olması.
- mAP@50-95 metriğinin 0.005 düzeyinde kalması; ancak bu değer, mevcut veri miktarı düşünüldüğünde beklenen bir sonuçtur.

Tablo III
PROJE SÜREÇ ANALİZİ: HIZLANDIRICI VE GEÇİKTİRİCİ ETKENLER

Süreç Aşaması	Tahmini Süre	Ana Etken
Veri Etiketleme	~1.5 gün	Geciktirici
Model Fine-Tuning	~0.5 gün	Nötr
LiDAR Pipeline Dev.	~1 gün	Geciktirici
GUI Entegrasyon	~0.5 gün	Geciktirici
Harvester Betiği	—	Hızlandırıcı (%40)
Toplam	~4 gün	—

VII. DONANIM ANALIZİ VE KISITLAMALAR

A. Geliştirme Donanımı

Proje, aşağıdaki tüketici sınıfı donanım üzerinde geliştirilmiş ve test edilmiştir:

- **İşlemci:** Intel Core i7-11800H (8 çekirdek, 16 iş parçacığı, maks. 4.6 GHz)
- **GPU:** NVIDIA RTX 3060 Laptop (12 GB GDDR6 VRAM, 3840 CUDA çekirdeği)
- **RAM:** 16 GB DDR4
- **İşletim Sistemi:** Windows 11

Bu konfigürasyon, statik kare analizinde tatmin edici sonuçlar üretmiştir. Ancak gerçek zamanlı video akışı üzerinde eş zamanlı 3B nokta bulutu işleme denendiğinde kare hızı (FPS) 5'in altına düşmüş ve akıcı bir kullanım deneyimi sağlanamamıştır. Bu nedenle projenin kapsamı, canlı video analizi yerine kare bazlı (frame-by-frame) statik analiz ile sınırlandırılmıştır.

B. H200 Sunucusunda Olası Senaryolar

NVIDIA H200 Tensor Core GPU içeren bir sunucuya erişim sağlanabilseydi projede gerçekleştirilebilecek ek özellikler şöyle sıralanabilir:

- **Gerçek Zamanlı 3B Derin Öğrenme:** 3B nokta bulutları üzerinde, basit DBSCAN kümeleme yerine Point-GNN veya VoxelNet gibi uçtan uca öğrenen mimariler çalıştırılabilir ve dinamik sahnelerde nesne tespiti gerçek zamana yakın bir hızda gerçekleştirilebilirdi.
- **Video Akışında Nesne Takibi:** Belirli bir LiDAR kameradan gelen canlı video akışında 120+ FPS hızında çok nesne takibi (multi-object tracking) uygulanabilirdi.
- **Büyük Ölçekli Eğitim:** Veri seti mevcut 80 kare yerine 10.000+ kare ile oluşturulabilir, bu da mAP@50 metriğinin %85 ve üzerine çıkmasına olanak tanıyabilirdi.
- **Kamera Projeksiyonu:** `Tr_velo_to_cam` matrisinin tüm dinamik kalibrasyon parametreleriyle doğru biçimde entegrasyonu sağlanarak LiDAR noktaları kamera görüntüsü üzerine piksel düzeyinde yansıtılabilirdi.

VIII. ZORLUKLAR VE TAMAMLANAMAYAN KISIMLAR

A. C++ Kütüphane Çakışmaları

Proje sürecinde en fazla zaman kaybettiren teknik sorun, Open3D'nin C++ tabanlı iş parçacığı modelinin PyQt6'nın Qt olay döngüsüyle uyumsuzluğundan kaynaklanan Windows fatal exception: code 0x8001010e hatasıydı. Bu hata, Open3D görselleştirme motorunun Qt'nin GUI iş parçacığından farklı bir COM iş parçacığı bağlamında çalışmaya çalışmasından kaynaklanmakta ve uygulamanın dondurulmasına ya da tamamen çökmesine yol açmaktaydı.

Çözüm, LiDAR işlemlerini `DataLoaderThread` içine alarak GUI iş parçacığından tam anlamıyla izole etmek ve iş parçacıkları arasındaki veri aktarımını Open3D nesneleri yerine serileştirilebilir NumPy dizileri aracılığıyla gerçekleştirmek olmuştur. Bu hata ayıklama süreci, yaklaşık yarım gün boyunca projeyi durdurmuştur.

Bunun yanı sıra, OpenMP ve Intel MKL kütüphanelerinin aynı Python ortamında hem Open3D hem de ONNX/PyTorch

tarafından birden fazla kez yüklenmesinden kaynaklanan `libi-omp5md.dll` çakışmaları da `main.py`'de ortam değişkenlerinin import öncesinde tanımlanmasıyla giderilmiştir.

B. Kamera Projeksiyonunun Tamamlanamaması

Projenin başlangıç hedefleri arasında yer alan, LiDAR noktalarının kamera görüntüsü üzerine piksel düzeyinde yansıtılması (camera projection) işlevi tam olarak gerçekleştirilememiştir. Bu işlevin doğru çalışabilmesi için Denklem 1'deki `Tr_velo_to_cam` matrisinin statik bir değer olarak değil, her çekim konfigürasyonuna özgü dinamik bir parametre olarak ele alınması gerekmektedir. KITTI kalibrasyon dosyaları bu parametreyi içermekle birlikte, sistemin sanal kamera bileşenine entegre edilmesi ve perspektif tutarlılığının sağlanması için gereken matematik hesaplamalarının eksiksiz uygulanması mevcut zaman kısıtları dahilinde tamamlanamamıştır.

IX. DERİN ÖĞRENME DERSİ BAĞLAMINDA DEĞERLENDİRME

Derin Öğrenme dersinin temel amacı, öğrencilere sınıf ortamında edinilen teorik bilgileri gerçek problemler üzerinde uygulamak, model eğitimi ve değerlendirme süreçlerine hâkimiyet kazandırmak ve derin öğrenmeyi pratikte ne anlam ifade ettiğini deneyimletmektir.

ODin3DPC bu amaçlarla şu noktalarda örtüşmektedir:

- Derste anlatılan transfer öğrenmesi kavramı, `yolo11n.pt` ağırlıklarından fine-tuning yoluyla doğrudan hayata geçirilmiştir.
- Makine öğrenmesi modeli (odin_brain) için öznetelik mühendisliği ve eğitim süreçleri adım adım uygulanmıştır.
- Precision, Recall ve mAP gibi model değerlendirme metrikleri, teoride öğrenilen biçimiyle yorumlanmış ve hangi kısıtların bu metrikleri düşürdüğü analiz edilmiştir.
- RANSAC ve DBSCAN gibi klasik yöntemlerin derin öğrenme modelleriyle birlikte nasıl hibrit bir mimari oluşturabileceği gösterilmiştir.

Bu proje, öğrenciye şu yetkinlikleri kazandırmıştır: büyük ölçekli bir modeli özel bir alana adapte etme, çok iş parçacıklı bir masaüstü uygulaması geliştirme, 3B geometri ve nokta bulutu işleme, veri etiketleme ve pipeline oluşturma, model performans analizi ve donanım kısıtlarını göz önünde bulundurarak mimari tasarım yapma.

X. UYGULAMA ALANLARI VE ENDÜSTRİYEL ENTEGRASYON

ODin3DPC'de geliştirilen bileşenler, aşağıdaki sektörlere doğrudan uyarlanabilir niteliktedir:

- **Akıllı Şehir Altyapısı:** Trafik kavşaklarına yerleştirilen LiDAR sensörleriyle araç ve yaya yoğunluğu gerçek zamanlı olarak analiz edilerek trafik ışığı kontrol sistemleri optimize edilebilir. 2B kamera görüntüleriyle birleştirilen bu sistem, plaka tanıma ve ihlal tespiti için de kullanılabilir.
- **Savunma Sanayii:** İnsansız kara araçlarında çevresel engel tespiti ve otonom manevra kararları için sensor

füzyon mimarisinin temel bileşenleri doğrudan uygulanabilir. Özellikle RANSAC ile zemin tespiti, engeli arazilerde dahi güvenli bir navigasyon tabanı sağlar.

- **Lojistik ve Depo Robotizasyonu:** Otonom forklift ve raftan-rafa robotic kolların insan çalışanları algılaması, güzergah planlaması ve dinamik engellerden kaçınması için 3B nokta bulutu tabanlı nesne tespiti kritik bir rol üstlenebilir.
- **Tarım Teknolojileri:** Otonom hasat robotlarının bitki sıralarını ve engellerini tanımlayabilmesi için aynı pipeline'ın özelleştirilmiş versiyonları kullanılabilir.
- **Tıbbi Görüntüleme:** YOLOv11'in fine-tuning mekanizması, tıbbi görüntüler üzerinde tümör veya anomali tespiti için benzer bir transfer öğrenmesi yaklaşımıyla adapte edilebilir.

XI. GELECEKTEKİ ÇALIŞMALAR VE GELİŞTİRME ÖNERİLERİ

A. Performans İyileştirmeleri

Kısa vadeli iyileştirmeler kapsamında TensorRT optimizasyonu ile YOLOv11 çıkarım süresinin 12 ms'den 3-5 ms seviyesine indirilmesi planlanmaktadır. Bu optimizasyon, video akışı üzerinde gerçek zamanlı analize kapı aralayacaktır. Orta vadede ise, `odin_brain.pkl` modelinin yerini alacak daha gelişmiş bir sınıflandırıcının daha büyük KITTI bölümleriyle eğitilmesi ve `Tr_velo_to_cam` entegrasyonunun tamamlanarak 2B ve 3B tespitlerin piksel düzeyinde örtüşmesi hedeflenmektedir.

B. Kullanıcı Arayüzü Geliştirmeleri

Arayüzün görsel kimliği, "Matrix" temalı bir fütüristik tasarıma dönüştürülerek teknoloji tutkunları için daha çekici bir deneyim sunulabilir. Arka plan renk paletinin siyah ve yeşil fosforlu tonlara taşınması, terminal tarzı yazı tiplerinin kullanılması ve nesneler üzerinde animasyonlu tarama efektleri bu tema kapsamında değerlendirilebilir.

C. Algoritma Geliştirmeleri

- DBSCAN'ın hiyerarşik versiyonu olan HDBSCAN, küme yoğunluklarının değişkenlik gösterdiği karmaşık sahnelerde daha doğru segmentasyon sunabilir.
- Kalman filtresi entegrasyonu, kareler arası nesne takibini (multi-object tracking) mevcut basit ID tabanlı hız tahmininin ötesine taşıyabilir.
- Daha fazla etiketli veriyle (5.000+ görüntü) yeniden eğitilen bir fine-tuned model, tüm sınıflarda anlamlı mAP değerlerine ulaşabilir.

XII. SONUÇ VE DEĞERLENDİRME

Yaptığım bu proje içerisinde raporun kendisinde de belirttiğim gibi birçok kazanım elde ettim. Bunlardan en kıymetlisi sabretmeyi öğrenmekti benim için, çünkü bu proje gerek model eğitiminde, gerek alınan hatalarda, gerek fine tuning yaparken görsel etiketleme yaparken ki harcanan sürede, gerek donanım bazlı elde etmek istediğim ama alamadığım sonuçlarda öğrendiğim yegane etmen sabır oldu. Bilgisayar gibi sanal bir ortamda iş yapmanın aslında ne kadar dikkat ve

sabır gerektirdiğini bu proje bana çok ciddi seviyede öğretti, bugüne dek bir çok proje geliştirmiştım ama Deep Learning dersi adına yaptığım bu proje hepsinden biraz biraz üstüne koyarak yapmam gereken bir proje oldu. Çok kez hata aldım, projenin sonuna yaklaştığım anlarda elimdeki tüm çalışan kodu kaybettim, kütüphane çatışması dolayısıyla elde edilemeyen sonuçlardaki hatayı kodlarda aradım, saatlerce uğraştım, pes etme noktasına geldim, projeyi tamamen sıfırlayıp yeniden yapma fikrine kapıldım ama bir anlık dahi olsa sakince oturup düşününce ve sabırlı davranınca hataların birer birer düzeldiğini görmem yanı sıra ne kadar stres yaparsam projenin o kadar zora girdiğini fark ettim. İnsanoğlu olarak her zaman her istediğimizi anında yapabilmeyi istemekteyiz, özellikle bunu sanal bir alemde yapıyorsak bu istek arşa çıkmakta; bu proje bana her zaman her istediğimin anında olmayacağını yanı sıra her istediğim şeyi de yapamayacağımı anlattı. Ben bu projeye başlarken çok daha farklı hayaller kurmuş ve ütöpik düşünmüştüm, bu düşünceler beni gerçeklikten uzaklaştırmış ve derin bir buhrana sürüklemişti. Proje başında yapmak istediklerimi gerçekleştiremedim belki evet ama en azından farklı yollar bularak kendimi o noktada geliştirebilme fırsatı sağladım. Bu projede bireysel davranmak gelecek iş hayatımda da bana çok ciddi bir kazanım sağladı, çünkü bu kadar kapsamlı bir projenin altından tek başıma kalkabilmeyi sağladım bu sayede kendime olan özgüvenim arttı. Teknik kısımlara değinmem gerekirse de anlatabileceklerim arasında; mükemmellik değil, sistem entegrasyonunun daha önemli olduğunun farkına vardım. Veri setinin küçüklüğü veya donanımın kısıtlılığı yüzünden mAP skorlarının düşük olmasının bir başarısızlık olmadığını, asıl başarının; birbirinden tamamen farklı iki yapıyı (YOLO ile 2D görüntü ve RANSAC/DBSCAN ile 3D nokta bulutu) aynı anda, eşzamanlı ve çokmeden çalıştıran bir sistem mimarisi (pipeline) kurmak olduğunu anladım. Bu proje anlatılan derslerin ne kadar teoride kaldığını, bizim bu dersimizin aksine pratiğe dökülmediğini bir kez daha benim gözlerimin önüne serdi. Sonuç olarak zoruyla, hayatı kolaylaştıran kodlarıyla bu proje bana gerek teknik, gerek manevi olarak çok ciddi kazanımlar kattı. Değerlendirmem gerekirse bu projeyi çok daha ileri seviyeye taşıyabilirdim belki; daha iyi donanım, daha fazla ekip arkadaşı, daha fazla geniş zaman ile belki de kusursuz bir proje ortaya koyabilirdim ama benim projemde yer alan kusurlar bundan sonraki projelerimde hata yapmamı engelleyecek uyarılar olacak ve her gün daha çok üstüne koymamı sağlayacak.

XIII. YARARLANILAN KAYNAKLAR

Projede başvuru temel akademik kaynaklar aşağıdaki gibi sıralanabilir. YOLO mimarisinin orijinal makalesi [1], nesne tespitini tek geçişli bir ağ problemi olarak formüle eden temel katkısıyla incelenmiş; özellikle gerçek zamanlı çıkarım hızını sağlayan ızgara tabanlı regresyon yaklaşımı dikkatle ele alınmıştır. KITTI benchmark makalesi [2], kalibrasyon matrislerinin nasıl oluşturulduğu ve LiDAR-kamera senkronizasyonunun nasıl gerçekleştirildiği açısından temel başvuru kaynağı olarak kullanılmıştır. Open3D kütüphanesi dokümantasyonu [5], RANSAC düzlem segmentasyonu ve DBSCAN kümeleme

API'lerinin doğru parametrelerle kullanılması için vazgeçilmez bir rehber niteliği taşımıştır.

KAYNAKLAR

- [1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [2] A. Geiger, P. Lenz, R. Urtasun, "Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Providence, RI, USA, 2012, pp. 3354–3361.
- [3] Ultralytics, "YOLOv11: Next-Generation Object Detection," Ultralytics Documentation, 2024. [Online]. Available: <https://docs.ultralytics.com>
- [4] C. R. Qi, L. Yi, H. Su, L. J. Guibas, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," *Adv. Neural Inf. Process. Syst. (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 5099–5108.
- [5] Q.-Y. Zhou, J. Park, V. Koltun, "Open3D: A Modern Library for 3D Data Processing," *arXiv preprint arXiv:1801.09847*, 2018. [Software v0.18], 2023. [Online]. Available: <http://www.open3d.org>
- [6] M. A. Fischler, R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, June 1981.
- [7] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Proc. 2nd Int. Conf. Knowl. Discov. Data Min. (KDD)*, Portland, OR, USA, 1996, pp. 226–231.